MEMORIA TÉCNICA DE INFERENCIA EN TIEMPO REAL • METODOLOGÍA KDD (8 ETAPAS)

🛡️ Arquitectura Interna de Ejecución de `app.py`: Trazabilidad de una Transacción desde la Interfaz hasta el Veredicto y Auditoría SQL

Explicación detallada paso a paso de los procesos matemáticos, reglas de decisión e ingeniería KDD que ejecuta el motor `FraudInferenceEngine` al evaluar una transacción en vivo.

Institución: Centro Universitario de Guadalajara (CUGDL) — UdeG

Autor: Roberto Alejandro Chagra Martínez

Programa: Lic. en Inteligencia Artificial y Ciencia de Datos

Módulos Auditados: `app.py`, `src/fraud_engine.py`, `models/*.joblib`

Enfoque KDD: Minería Híbrida (Isolation Forest + Random Forest SMOTE + Reglas R1–R5)

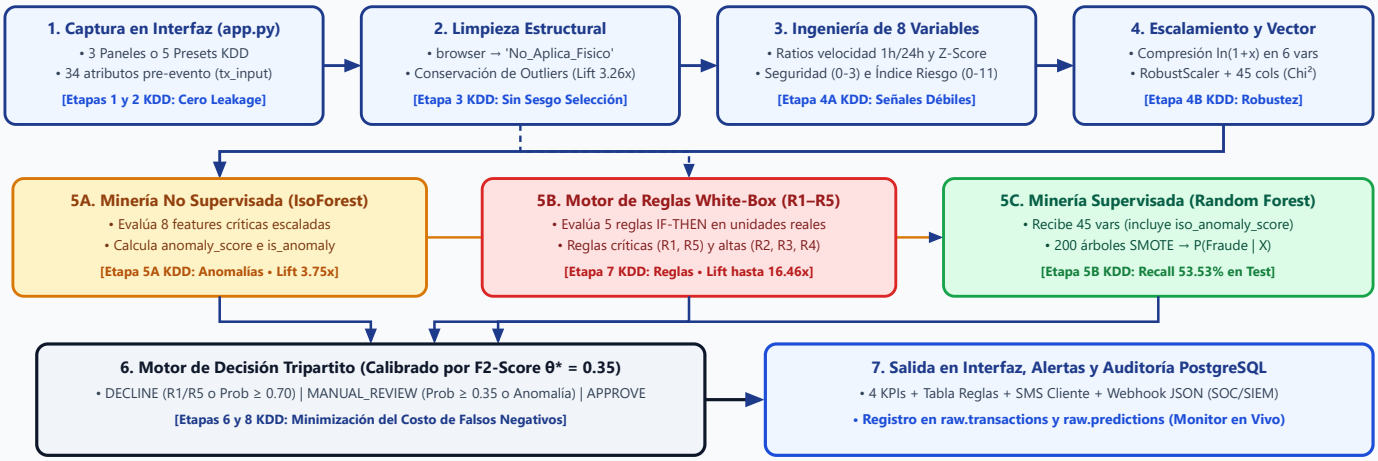
Persistencia SQL: PostgreSQL (`raw.transactions` y `raw.predictions`)

1. Del Descubrimiento KDD Offline a la Inferencia en Tiempo Real (Online)

En la metodología **KDD (*Knowledge Discovery in Databases*)** aplicada a sistemas antifraude bancarios, la arquitectura se divide en dos fases sincronizadas:

1. **Fase Offline de Descubrimiento y Entrenamiento (`src/train_and_save.py`):** Analiza el repositorio histórico de **100,000 transacciones** (97.945% legítimas vs. 2.055% fraudulentas, razón 47.66 : 1) recorriendo las Etapas 1 a 7 de KDD para auditar la fuga de información (*Data Leakage*), diagnosticar valores atípicos, construir 8 características financieras, seleccionar las 45 variables óptimas vía Chi-Cuadrado (χ^2), ajustar los modelos y exportar los artefactos serializados en `models/`.
2. **Fase Online de Explotación Interactiva (`app.py` + `src/fraud_engine.py` — Etapa 8 KDD):** Cuando el usuario ingresa una transacción en la interfaz de Streamlit y pulsa **"Evaluar Transacción & Ejecutar Pipeline de Fraude"**, el motor `FraudInferenceEngine.predict(tx_input)` reproduce **en milisegundos** cada transformación matemática y regla del proceso KDD sobre esa transacción individual para emitir el dictamen, simular alertas SMS/Webhook y registrar la auditoría en PostgreSQL.

Diagrama de Arquitectura Interna al Evaluar una Transacción en `app.py`



2. Trazabilidad Interna Paso a Paso en `app.py` y `src/fraud_engine.py`

Paso 1: Carga en Caché de Artefactos KDD (`load_engine()`)

Inicialización en RAM

Al iniciar `app.py`, el decorador `@st.cache_resource` instancia una sola vez en memoria compartida la clase `FraudInferenceEngine` definida en `src/fraud_engine.py`. El constructor carga desde `models/` cuatro artefactos matemáticos inmutables obtenidos durante el entrenamiento KDD:

- `scaler.joblib` (**RobustScaler**): Almacena la mediana y el rango intercuartílico (IQR = $Q_3 - Q_1$) de las 26 variables numéricas calculadas exclusivamente sobre las 80,000 filas de entrenamiento.
- `isolation_forest.joblib` (**IsolationForest**): Bosque no supervisado de 150 árboles ajustado con tasa de contaminación del 3.5% sobre las 8 variables de mayor sensibilidad conductual.
- `fraud_classifier.joblib` (**RandomForestClassifier**): Ensamble supervisado de 200 árboles entrenado sobre el espacio balanceado sintéticamente con `SMOTE(sampling_strategy=0.30)`.
- `model_columns.json`: Vector canónico ordenado de las **45 características óptimas** seleccionadas mediante la prueba Chi-Cuadrado (χ^2) e importancia Gini.

Paso 2: Captura del Vector Transaccional y Blindaje Anti-Leakage (`tx_input`)

Etapas 1 y 2 KDD

Cuando el usuario selecciona un escenario predefinido en la barra lateral o ajusta manualmente los controles de las tres columnas (*Detalles Financieros*, *Ratios de Velocidad y Ubicación*, y *Factores de Seguridad y Entorno*), `app.py` ensambla el diccionario `tx_input` con los atributos observables en el instante exacto del pago (t_0).

Cumplimiento estricto de la Etapa 2 KDD (Prevención de *Data Leakage*): El formulario y el diccionario `tx_input` excluyen por diseño las variables post-evento (`fraud_type` y `fraud_amount_lost`, que en la vida real solo se conocen semanas después tras la investigación de un contracargo) y excluyen los identificadores UUID (`transaction_id`, `customer_id`, `card_id`, `merchant_id`) del vector predictivo. El `transaction_id` generado (`TXN-XXXXXXXX`) se utiliza únicamente para trazabilidad en alertas y llave primaria en PostgreSQL.

Paso 3: Limpieza Estructural e Imputación en Vuelo (`preprocess_and_feature_engineering`)

Etapa 3 KDD

Al presionar **"Evaluar Transacción"**, se invoca `engine.predict(tx_input)`. El primer método interno es `preprocess_and_feature_engineering(raw_tx)`, el cual aplica las dos decisiones científicas de la Etapa 3 KDD:

1. **Imputación Semántica sin Sesgo de Selección (`browser_clean`):** En la Etapa 3 KDD se demostró que el 100% de los 41,709 valores nulos en `browser` correspondían a transacciones presenciales (`In-store` y `ATM`). En vuelo, si la transacción ocurre en terminal física o carece de navegador, el motor asigna la categoría estructural `'No_Aplica_Fisico'`.
2. **Principio *Outlier ≠ Error* (Conservación de Extremos):** Mediante el criterio 1.5 × IQR de Tukey se comprobó que los valores atípicos en monto (`> $185.75`) o velocidad (`> 2.5 txs/h`) concentran hasta **3.26 veces más probabilidad de fraude**. Por ello, el motor no recorta (*clipping*) ni descarta valores extremos ingresados en la interfaz; los preserva íntegros para su evaluación.

Paso 4: Ingeniería de las 8 Características Derivadas en Tiempo Real

Etapa 4A KDD

Para superar el problema de las **señales débiles** (donde el defraudador imita compras legítimas en variables individuales), el motor calcula instantáneamente 8 nuevas variables relacionales y de perfil conductual:

#	Variable Derivada	Fórmula Matemática Calculada en Vuelo	Patrón de Fraude que Captura (Dominio KDD)
1	`velocity_tx_ratio_1h_24h`	`transactions_last_1h / (transactions_last_24h + 1.0)`	Aceleración súbita de frecuencia (ataques automatizados de *carding* concentrados en la última hora).
2	`velocity_amt_ratio_1h_24h`	`amount_last_1h / (amount_last_24h + 1.0)`	Concentración monetaria aguda en los últimos 60 minutos respecto al gasto diario.
3	`credit_utilization_ratio`	`transaction_amount / (credit_limit + 1.0)`	Intento de agotamiento abrupto de la línea de crédito (*bust-out fraud*) en un solo cargo.
4	`zscore_amount_30d`	`(transaction_amount − avg_30d) / (std_30d + 1.0)`	Desviación estandarizada (σ) respecto al comportamiento histórico de 30 días del cliente.
5	`security_factors_count`	`card_present + used_chip + used_pin` ∈ {0, 1, 2, 3}	Nivel de blindaje físico de la operación (0 = sin factores físicos; 3 = Tarjeta + Chip EMV + PIN).
6	`is_night_tx`	`1` si `transaction_hour ≤ 4`, sino `0`	Operación en ventana nocturna crítica (00:00–04:59 h), horario preferido en ataques internacionales.
7	`is_new_account`	`1` si `account_age_days ≤ 90`, sino `0`	Vulnerabilidad por cuenta de reciente apertura (≤ 3 meses), asociada a identidades sintéticas.
8	`composite_risk_index`	Suma ponderada de 10 señales concurrentes (0 a 11 pts)	Suma +2 por país de alto riesgo y +1 por cada señal concurrente (internacional, sin tarjeta, ≥ 3 txs/1h, ≥ 2 contracargos, ≥ 2 declinaciones, madrugada, cuenta nueva, ratio monto > 4.8×, distancia > 800 km).

Paso 5: Evaluación de las 5 Reglas de Decisión White-Box (`evaluate_business_rules`)

Etapa 7 KDD

Sobre el DataFrame en sus **unidades físicas originales** (dólares, kilómetros, horas), el motor evalúa las 5 reglas deterministas `IF-THEN` inducidas del Árbol de Decisión Explicable en la Etapa 7 KDD:

Regla	Condición Lógica Exacta (`IF`)	Severidad	Precisión KDD	Factor Lift (vs 2.05%)
R1	`high_risk_country == 1 AND card_present == 0 AND transactions_last_1h ≥ 3`	CRITICAL	**33.83%**	**16.46x** (Bloqueo directo)
R2	`high_risk_country == 1 AND account_age_days ≤ 90`	HIGH	**23.19%**	**11.28x** (Alerta alta)
R3	`transaction_hour ≤ 4 AND international_transaction == 1 AND card_present == 0`	HIGH	**18.70%**	**9.10x** (Retención 2FA)
R4	`amount_vs_avg_ratio > 4.8 AND num_chargebacks_ever ≥ 2 AND card_present == 0`	HIGH	**24.12%**	**11.74x** (Alerta alta)
R5	`composite_risk_index ≥ 5` *(acumulación de 5 o más señales débiles simultáneas)*	CRITICAL	**20.11%**	**9.79x** (Bloqueo directo)

Paso 6: Compresión Logarítmica, Escalamiento Robusto y Vector de 45 Variables

Etapa 4B KDD

Antes de invocar a los modelos de Machine Learning, `src/fraud_engine.py` transforma las características al espacio matemático exacto de entrenamiento:

1. **Compresión Logarítmica:** Aplica $x_{log} = \ln(1 + \max(0, x))$ sobre las 6 variables con asimetría positiva extrema (`transaction_amount` , `amount_last_1h` , `amount_last_24h` , `distance_from_home_km` ,

`distance_from_last_tx_km`, `amount_vs_avg_ratio`).

2. **Estandarización con `RobustScaler` (`scaler.joblib`):** Transforma las 26 variables numéricas mediante estadísticos insensibles a valores extremos:

$$z_i = [\, x_i - \text{Mediana}_{train}(X_i) \,] \;/\; \text{IQR}_{train}(X_i)$$

3. **Reconstrucción Determinística del Vector Canónico (`model_columns.json`):** Ensambla en orden exacto las **45 variables** (26 numéricas escaladas + `iso_anomaly_score` + 18 binarias/dummies seleccionadas por **Chi-Cuadrado χ^2**). A diferencia de `pd.get_dummies(drop_first=True)` —que fallaría al evaluar una sola fila—, el motor asigna 1 o 0 comparando directamente el valor de cada campo categórico contra el nombre de la columna esperada.

Paso 7: Ejecución Acoplada de la Minería de Datos Híbrida (No Supervisada + Supervisada)

Etapa 5 KDD

- **7A. Aislamiento No Supervisado de Anomalías (`isolation_forest.joblib`):** Evalúa las 8 variables críticas escaladas (`self.iso_features`) en los 150 árboles aleatorios de aislamiento. Calcula el puntaje continuo `iso_score = -iso_forest.score_samples(...)` y el indicador booleano `is_anomaly` (`predict == -1`). En validación KDD, las transacciones aisladas por este bosque presentan una tasa de fraude de **7.07% vs. 1.88%** en las normales (**Lift = 3.75x**). Este puntaje se inyecta como la variable #27 (`iso_anomaly_score`) hacia el clasificador supervisado.
- **7B. Clasificación Supervisada con Random Forest + SMOTE (`fraud_classifier.joblib`):** El vector completo de 45 características ingresa a los 200 árboles del Random Forest (entrenado con sobremuestreo sintético SMOTE al 30% y pesos balanceados, alcanzando un **Recall de 53.53%** y **ROC-AUC de 0.8325**). El modelo calcula la probabilidad continua de fraude promediando las predicciones de sus 200 árboles: `fraud_prob = classifier.predict_proba(features_for_model)[:, 1][0]`.

Paso 8: Motor de Decisión Tripartito y Calibración Sensible al Costo (F_2-Score)

Etapas 6 y 8 KDD

En la Etapa 6 KDD se demostró la **Paradoja del Accuracy** (un modelo que predice siempre "legítima" logra 97.95% de exactitud pero 0% de fraudes detectados) y se calibró el umbral óptimo del Random Forest maximizando el ***F_2*-Score** (que pondera el *Recall* el doble que la *Precision* debido al alto costo financiero de un Falso Negativo), obteniendo **$\theta^* = 0.35$**:

$$F_2 = (5 \cdot \text{Precision} \cdot \text{Recall}) \;/\; (4 \cdot \text{Precision} + \text{Recall}) \implies \text{Umbral Óptimo Calibrado: } \theta^* = 0.35$$

Combinando el umbral θ* = 0.35, el Isolation Forest y las reglas R1–R5, el motor aplica tres niveles de respuesta:

Veredicto	Regla de Activación en `src/fraud_engine.py`	Acción Operativa y Alerta Despachada
🔴 DECLINE	`critical_rule_triggered (R1 o R5)` **OR** `fraud_probability ≥ 0.70`	**Bloqueo preventivo inmediato:** Rechaza el cargo, congela temporalmente la tarjeta y envía SMS crítico (código 8800) + Webhook `CRITICAL`.
⚠️ MANUAL_REVIEW	`0.35 ≤ fraud_probability < 0.70` **OR** `is_anomaly == True` **OR** `any_rule_triggered (R2, R3, R4)`	**Retención / Autenticación 2FA:** Solicita aprobación del cliente con Token Digital en la App Bancaria y envía alerta `WARNING` al SOC.
✅ APPROVE	`fraud_probability < 0.35` **AND** `is_anomaly == False` **AND** `0 reglas activadas`	**Aprobación sin fricción:** Autoriza el pago en línea y registra la operación como legítima de bajo riesgo (`LOW`).

Paso 9: Despacho de Alertas Multi-Canal y Persistencia de Auditoría en PostgreSQL

Etapa 8 KDD (MLOps)

- **Renderizado Visual y Simulación Multi-Canal (Secciones 2 y 3 de `app.py`):** Despliega el banner semafórico, las 4 tarjetas KPI, la tabla de evaluación de las 5 reglas y el método `_build_alert_payload()` que construye en vivo el **SMS bancario al cliente** y el **payload JSON del Webhook** para sistemas SIEM/SOC/Slack.
- **Persistencia y Monitor SQL en Vivo (Sección 4 y Pestaña 2 de `app.py`):** Al presionar **"Registrar esta Transacción y su Predicción en PostgreSQL"**, `app.py` inserta los datos pre-evento en `raw.transactions` y la evaluación completa en `raw.predictions` (`transaction_id`, `scoring_timestamp`, `fraud_probability`, `model_prediction`,

> `anomaly_score`, `recommended_action`), alimentando en tiempo real la tabla de auditoría y el gráfico de distribución de la Pestaña 2 junto con los micro-lotes procesados cada 5 minutos por Apache Airflow.

3. Comparativa Numérica Real: Trazabilidad Interna de 3 Escenarios de `app.py`

La siguiente tabla documenta los valores exactos calculados en cada etapa interna de `FraudInferenceEngine` cuando se ejecutan tres de los escenarios preconfigurados en `app.py`:

Etapa KDD / Variable Interna	🟢 Escenario 1: Transacción Legítima Habitual	🔴 Escenario 2: Fraude Crítico Internacional (R1)	🟡 Escenario 3: Desviación Anómala (Isolation Forest)
1. Entrada: Monto, Comercio y País	`$35.50 USD` • Supermarket (USA)	`$2,950.00 USD` • Online Retail (Nigeria)	`$1,450.00 USD` • Electronics (Romania)
2. Entrada: Hora, Canal y Dispositivo	`14:00 h` • In-store (POS Terminal)	`03:00 h` • Online (Desktop)	`16:00 h` • Online (Mobile)
3. Etapa 3 KDD: `browser_clean`	`'No_Aplica_Fisico'` (imputado)	`'Chrome'`	`'Safari'`
4. Etapa 4 KDD: `velocity_tx_ratio_1h_24h`	`0.333` (1 tx en 1h / 2 en 24h)	`0.500` (4 txs en 1h / 7 en 24h)	`0.400` (2 txs en 1h / 4 en 24h)
5. Etapa 4 KDD: `credit_utilization_ratio`	`0.006` (0.6% del límite)	**`0.983` (98.3% del límite)**	`0.181` (18.1% del límite)
6. Etapa 4 KDD: `zscore_amount_30d`	`−0.346 σ` (dentro del hábito)	**`+151.842 σ` (ruptura extrema)**	**`+87.188 σ` (desviación severa)**
7. Etapa 4 KDD: `security_factors_count`	`3 / 3` (Tarjeta + Chip + PIN)	`0 / 3` (Sin factores físicos)	`0 / 3` (Sin factores físicos)
8. Etapa 4 KDD: `composite_risk_index`	`0 señales` (riesgo nulo)	`11 señales` (máximo posible)	`4 señales` (bajo el corte de R5)
9. Etapa 5A KDD: `anomaly_score` (IsoForest)	`0.4992` (`is_anomaly = False`)	**`0.7559` (`is_anomaly = True`)**	**`0.7134` (`is_anomaly = True`)**
10. Etapa 5B KDD: `fraud_probability` (RF)	`0.11%` (`0.0011`)	`64.87%` (`0.6487`)	**`77.30%`** (`0.7730`)
11. Etapa 7 KDD: Reglas Violadas	Ninguna (`[]`)	`['R1', 'R2', 'R3', 'R4', 'R5']`	Ninguna (`[]` — evade reglas fijas)
12. Etapa 8 KDD: Veredicto Final	✅ APPROVE	🔴 DECLINE	🔴 DECLINE

> 💡 **Demostración del Valor de la Minería Híbrida KDD (Por qué el Escenario 3 es bloqueado aunque no viola ninguna regla R1–R5):**
> En el **Escenario 3 (Desviación Anómala)**, el defraudador evade deliberadamente las 5 reglas estáticas de negocio: opera de día (`16:00 h`, evade R3), desde una cuenta antigua de `1,200 días` (evade R2), sin contracargos previos (evade R4), con solo 2 transacciones en la última hora (evade R1) y con un índice compuesto de `4 puntos < 5` (evade R5). Un motor bancario tradicional basado únicamente en reglas habría aprobado el cargo. Sin embargo, en `app.py`, el modelo no supervisado **Isolation Forest (Etapa 5A KDD)** detecta la atipicidad combinada del monto (`zscore = +87.19σ`) y la distancia (`1,250 km`), marcando `is_anomaly = True` (`score = 0.7134`) e inyectando esta señal al **Random Forest (Etapa 5B KDD)**, el cual eleva la probabilidad de fraude a **77.30% (≥ 0.70)** y bloquea preventivamente el ataque con veredicto `DECLINE`.